

Linux Commands

Permissions, Users and Groups

Why "Permission denied" happens, and what to do about it.

Bhuvnesh Verma

github.com/MasterBhuvnesh/notebooks

Contents

1	File Permissions	2
1.1	Permission Types	2
1.2	Who Gets Permissions?	2
1.3	Example: 755	3
1.4	Example: 700	3
1.5	Example: 777	3
1.6	Example: 644	3
1.7	What Does Execute Mean for a Directory?	3
1.8	Viewing Permissions	3
1.9	Changing Permissions	4
1.10	Common Modes You Will Use	4
2	Users, Groups and Ownership	5
2.1	Viewing the Current User	5
2.2	What is a User?	5
2.3	What is a Group?	5
2.4	Managing Groups	5
2.5	Deleting a User	6
2.6	Where Users Are Stored	6
2.7	Where Passwords Are Stored	6
2.8	What is Root?	6
2.9	Ownership of Files	6
2.10	Directory Permissions Example	6
2.11	Real Company Example	7
2.12	Sudo Access	7
2.13	System Users vs Human Users	7
2.14	Process Ownership	7
2.15	Useful Admin Commands	8
2.16	How This Applies to Docker, Nginx and PostgreSQL	8

1. File Permissions

In Linux, **permissions (modes)** control who can **read, write, or execute** a file or directory. When you use:

```
mkdir -m 755 myapp
```

the 755 is the **permission mode** assigned to the directory.

1.1 Permission Types

There are three permission bits:

Permission	Symbol	Value
Read	r	4
Write	w	2
Execute	x	1

The values are added together:

Permission Set	Calculation	Value
---	0	0
--x	1	1
-w-	2	2
-wx	2+1	3
r--	4	4
r-x	4+1	5
rw-	4+2	6
rwX	4+2+1	7

1.2 Who Gets Permissions?

Linux divides users into three groups:

Group	Meaning
User (u)	Owner of the file
Group (g)	Users in the file's group
Others (o)	Everyone else

A permission like 755 means:

```

7   5   5
|   |   |
|   |   +-- Others
|   +----- Group
+----- Owner
```

1.3 Example: 755

Break it down:

$$\begin{aligned} 7 &= rwx = 4+2+1 \\ 5 &= r-x = 4+1 \\ 5 &= r-x = 4+1 \end{aligned}$$

Result: `rw-r-xr-x`. Meaning:

User Type	Permissions
Owner	Read, Write, Execute
Group	Read, Execute
Others	Read, Execute

1.4 Example: 700

```
7 = rwx
0 = ---
0 = ---
```

Result: `rwX-----`. Only the owner can access it. Useful for:

```
mkdir -m 700 secrets
```

1.5 Example: 777

Result: **rw-rw-rwx**. Everyone can read, write and execute.

Warning: Usually avoided, because anyone can modify the contents.

1.6 Example: 644

Common for files.

```
6 = rw-
4 = r--
4 = r--
```

Result: `rw-r--r--`. The owner can read and write; everyone else can only read.

1.7 What Does Execute Mean for a Directory?

For a **file**, **x** means the file can be run as a program or script. For a **directory**, **x** means you can enter it, that is `cd` into it.

```
chmod 600 mydir
```

That gives `rw-----`. You can see the directory, but you **cannot** `cd` into it, because execute permission is missing. To enter a directory, `x` permission is required.

1.8 Viewing Permissions

1s -1

Example output:

```
drwxr-xr-x 2 bhuvnesh users 4096 Aug 15 project
```

Breakdown:

```
d rwx r-x r-x
| |  |  |
| |  |  +-- Others
| |  +----- Group
| +----- Owner
+----- Directory
```

1.9 Changing Permissions

Numeric method:

```
chmod 755 project
chmod 700 secrets
chmod 644 file.txt
```

Symbolic method:

```
chmod u+x script.sh  # add execute permission for the owner
chmod g+w file.txt   # add write permission for the group
chmod o-r file.txt   # remove read permission for others
```

1.10 Common Modes You Will Use

Mode	Meaning	Typical Use
755	rwxr-xr-x	Application directories
700	rwX-----	Private directories
644	rw-r--r--	Regular files
600	rw-----	Secrets, keys, passwords
777	rwXrwxrwx	Rarely recommended

Real-world examples

```
mkdir -m 755 backend
mkdir -m 700 .ssh
touch app.js
chmod 644 app.js
chmod 600 id_rsa
```

For software development you will most often see:

- **Directories:** 755
- **Source code files:** 644
- **Shell scripts:** 755
- **SSH private keys and .env files:** 600 or 400

2. Users, Groups and Ownership

To really understand Linux permissions, you need to understand **users**, **groups** and **ownership** first. Linux is a multi-user operating system. On a company server:

```
ubuntu
+-- bhuvnesh
+-- rohan
+-- alice
+-- john
```

Each person has their own account.

2.1 Viewing the Current User

```
whoami
# bhuvnesh

id
# uid=1000(bhuvnesh) gid=1000(bhuvnesh) groups=1000(bhuvnesh),27(sudo)
```

- UID = User ID
- GID = Group ID
- Groups = the groups this user belongs to

2.2 What is a User?

A user is an account that can log in, own files, run processes and hold permissions.

```
sudo useradd john    # create a user
sudo passwd john     # set the password
```

2.3 What is a Group?

A group is a collection of users.

```
developers
+-- bhuvnesh
+-- rohan
+-- alice
```

Instead of assigning permissions to each user, assign them to the group.

2.4 Managing Groups

```
sudo groupadd developers    # create a group
cat /etc/group              # check it exists

sudo usermod -aG developers john # add a user (-a append, -G group)
groups john
# john : john developers

sudo gpasswd -d john developers # remove a user from the group
sudo groupdel developers       # delete the group
```

2.5 Deleting a User

```
sudo userdel john      # delete the user
sudo userdel -r john   # delete the user and their home directory
```

2.6 Where Users Are Stored

Linux stores users in `/etc/passwd`:

```
cat /etc/passwd
```

```
john:x:1001:1001::/home/john:/bin/bash
```

The fields are: username, password placeholder, UID, GID, comment, home directory, shell.

2.7 Where Passwords Are Stored

```
sudo cat /etc/shadow
```

Only root can read it. It contains hashed passwords.

2.8 What is Root?

Linux has a superuser called `root`. Root can create and delete users, modify any file, install software and shut the server down.

```
sudo su
# or
su -

whoami
# root
```

2.9 Ownership of Files

```
touch app.js
ls -l
# -rw-r--r-- 1 bhuvnesh developers 0 Aug 15 app.js
```

Here the owner is `bhuvnesh` and the group is `developers`.

```
sudo chown john app.js      # change the owner
sudo chgrp developers app.js # change the group
sudo chown john:developers app.js # change both at once
```

2.10 Directory Permissions Example

```
mkdir project
chmod 770 project
```

With owner `bhuvnesh` and group `developers`, 770 means `rxwxrwx---`:

Who	Access
Owner	Full
Group	Full
Others	None

So anyone in **developers** can use the directory.

2.11 Real Company Example

```

developers      designers
+-- bhuvnesh    +-- john
+-- rohan       +-- sara
+-- alice

```

Create a shared directory:

```

sudo mkdir /shared-code
sudo chown root:developers /shared-code
sudo chmod 770 /shared-code

```

Developers can access it; designers cannot.

2.12 Sudo Access

Not every user can become root. On Ubuntu, users allowed to run **sudo** belong to the **sudo** group.

```

groups bhuvnesh
# bhuvnesh sudo

sudo usermod -aG sudo john  # now john can run: sudo apt update

```

2.13 System Users vs Human Users

Listing `/etc/passwd` shows entries like **root**, **daemon**, **www-data**, **mysql**, **postgres**, **nginx**, **ubuntu** and **bhuvnesh**. Not all of them are humans.

User	Purpose
root	Administrator
mysql	MySQL service
postgres	PostgreSQL service
nginx	Nginx server
www-data	Apache / Nginx web apps

These are called **service accounts**.

2.14 Process Ownership

Every running process belongs to a user.

```
ps aux
```



```
USER      PID
root      123
postgres  456
nginx     789
```

PostgreSQL runs as `postgres` and Nginx runs as `nginx`. This improves security.

2.15 Useful Admin Commands

Command	Description	Example
<code>useradd</code>	Create a user, with a home directory	<code>sudo useradd -m john</code>
<code>passwd</code>	Set a user's password	<code>sudo passwd john</code>
<code>groupadd</code>	Create a group	<code>sudo groupadd developers</code>
<code>usermod</code>	Add a user to a group	<code>sudo usermod -aG developers john</code>
<code>chown</code>	Change the owner of a file	<code>sudo chown john file.txt</code>
<code>chgrp</code>	Change the group of a file	<code>sudo chgrp developers file.txt</code>
<code>userdel</code>	Delete a user and their home directory	<code>sudo userdel -r john</code>
<code>groupdel</code>	Delete a group	<code>sudo groupdel developers</code>
<code>whoami</code>	Show the current user	<code>whoami</code>
<code>id</code>	Show user and group IDs	<code>id</code>

2.16 How This Applies to Docker, Nginx and PostgreSQL

When you run `docker run nginx`, Nginx inside the container often runs as `nginx`, not `root`. When PostgreSQL starts, `postgres` owns the database files. When GitHub Actions deploys code, `ubuntu` or a dedicated deployment user may own the files.

Understanding **users, groups, ownership and permissions** is one of the most important Linux concepts, because it explains why commands sometimes fail with:

```
Permission denied
```

Most of the time the current user is not the owner, is not in the required group, or lacks the necessary permission bits (**r**, **w**, **x**).